

Program Analysis for Quantitative-reachability Properties

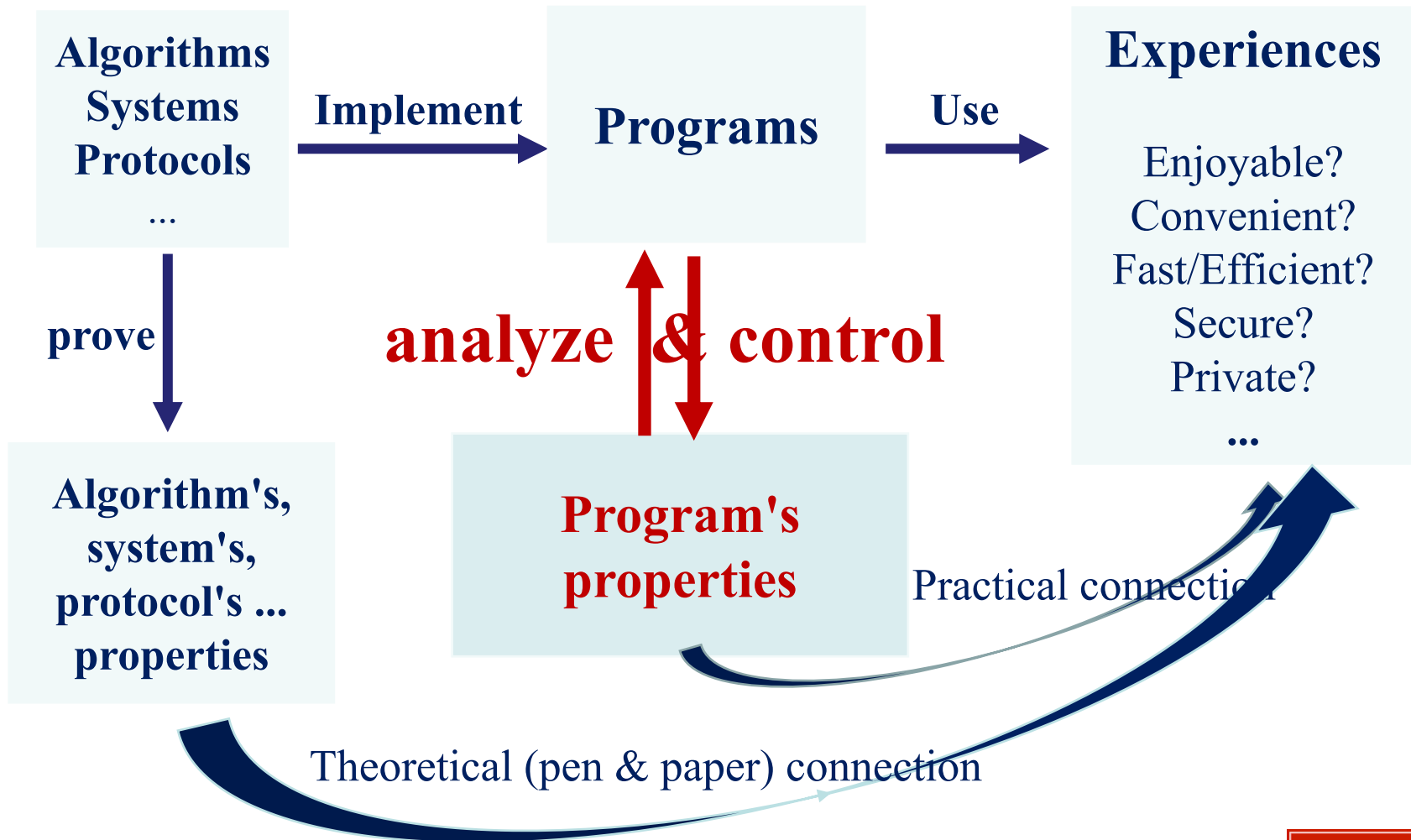
Jiawen Liu

Advised by: Marco Gaboardi

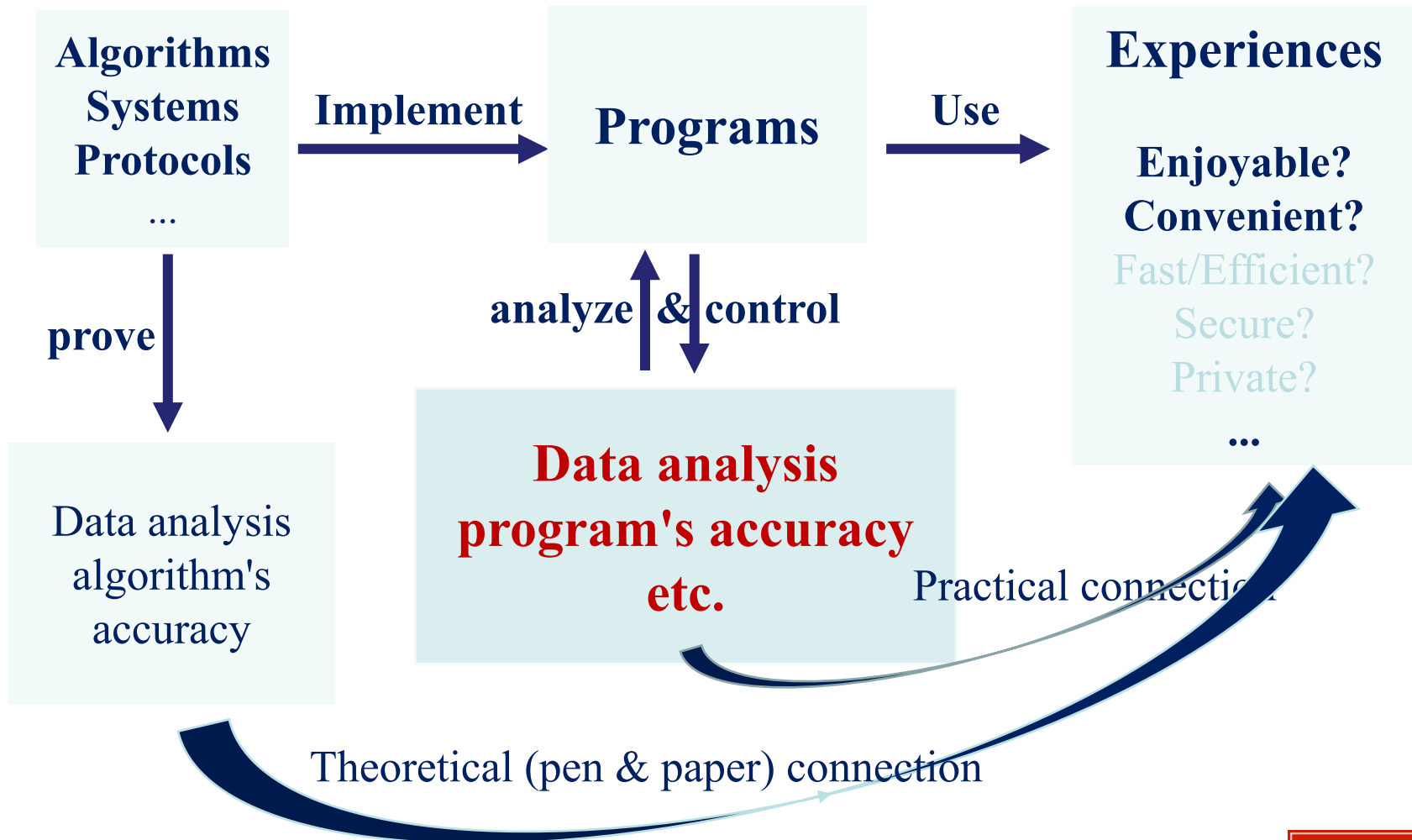
Outline

- Introduction
- Program Analysis for Adaptive Data Analysis
 - Introduction
 - QWhile Language
 - Adaptivity Formalization
 - Program Analysis Algorithm
- Path-sensitive Reachability-bound Analysis (In Submission)
 - Introduction
 - Language Model
 - Reachability-bound
 - Path-sensitive Reachability-bound Algorithm
- Future Works

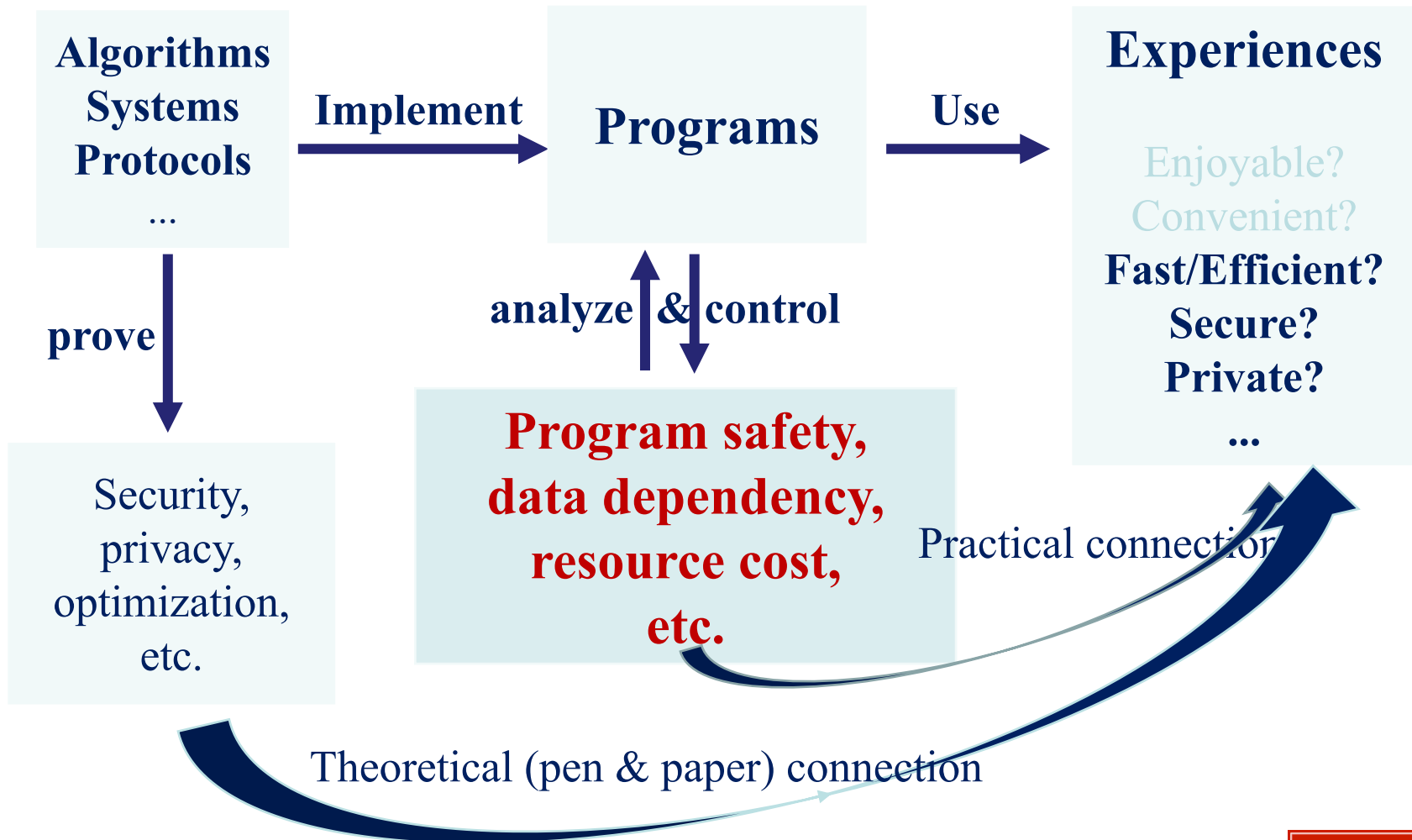
Introduction – program analysis background



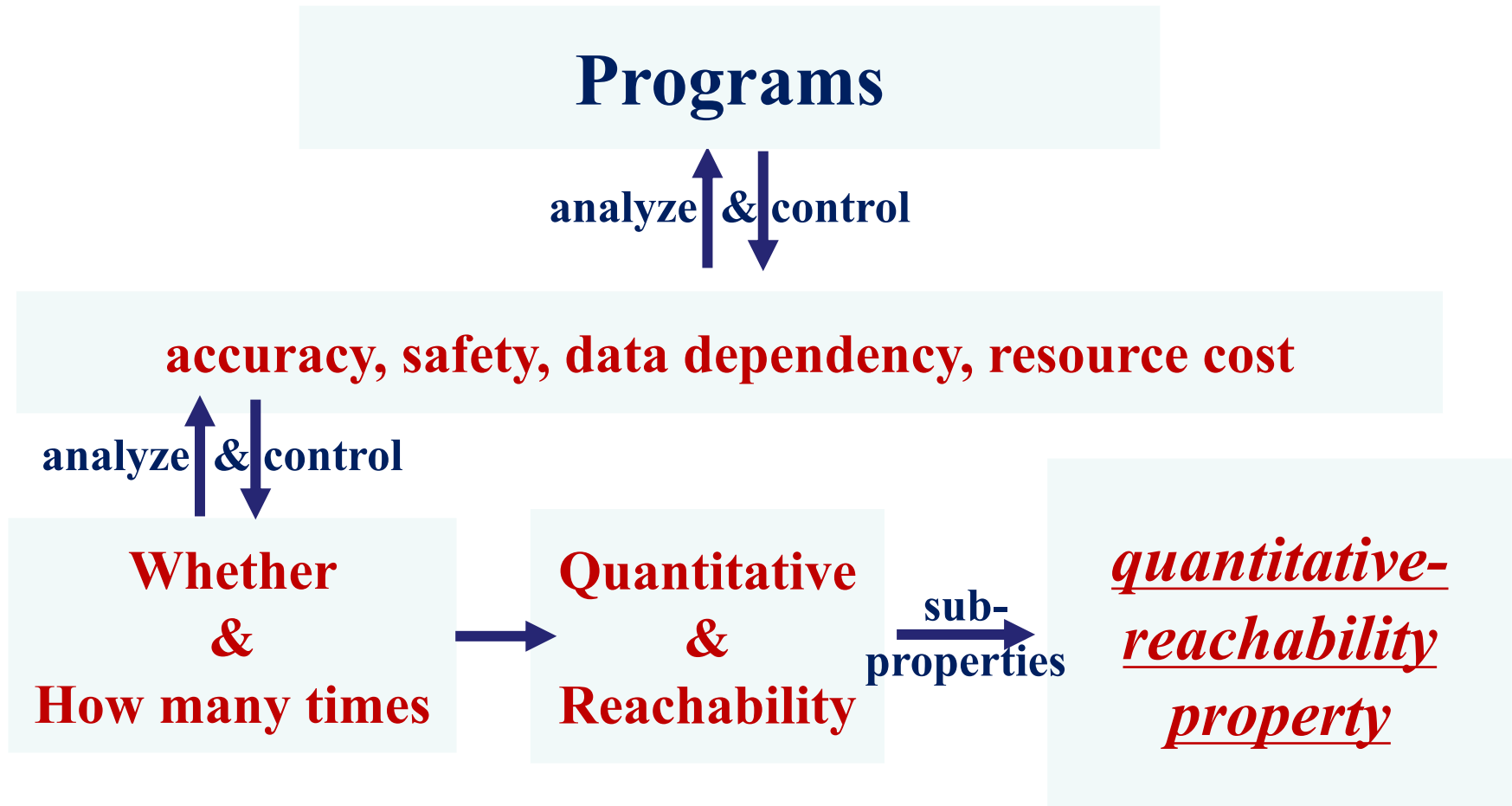
Introduction – program analysis background



Introduction – program analysis background



Introduction – quantitative-reachability property



Introduction – challenge : property formalization challenge

quantitative-reachability properties



**Variable/Data/Code
reachability,
commutative,
liveness, etc**

**Quantitative
&
Reachability**

**Resource cost /
Worst-case execution
time / Complexity, etc**

Introduction – *new methodology*

Semantics-based quantitative-reachability formalization

- Define sub-property "*quantitative*" & "*reachability*",
- ➔
- Define new *structure/model* encoding the two sub-properties,
- ➔
- Define the "*quantitative-reachability*" as a hyper-property over the model

Introduction – challenge : program analysis challenge

quantitative-reachability properties



Control-flow, data-flow, liveness, dependency analysis, etc.

**Quantitative
&
Reachability**

Resource cost, worst-case execution time, complexity estimation

reachability



overall quantity

Introduction – *new methodology*

➔ *Analyze the quantitative-reachability integrally:*

Analyze sub-properties "*quantitative*" & "*reachability*"

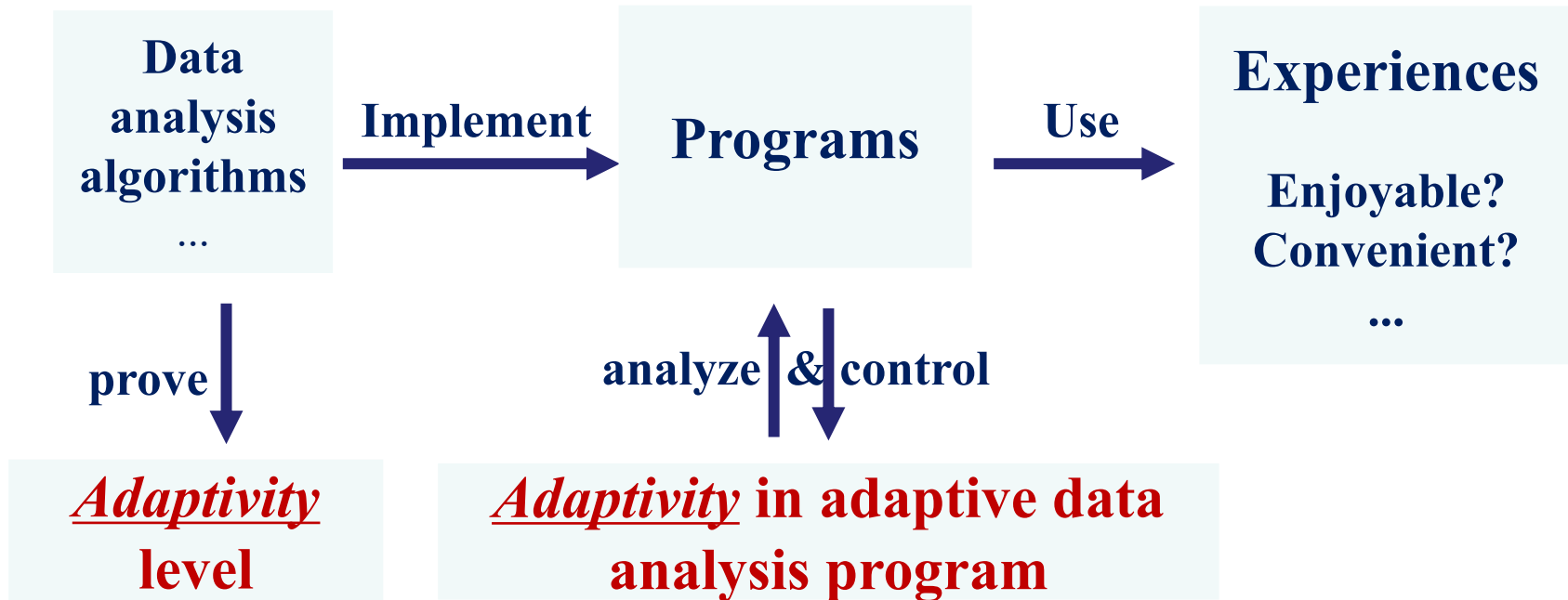


Estimate the new *structure/model*



Analyze the hyper-property "*quantitative-reachability*"

Introduction – first quantitative-reachability property



Introduction – *second quantitative-reachability property*



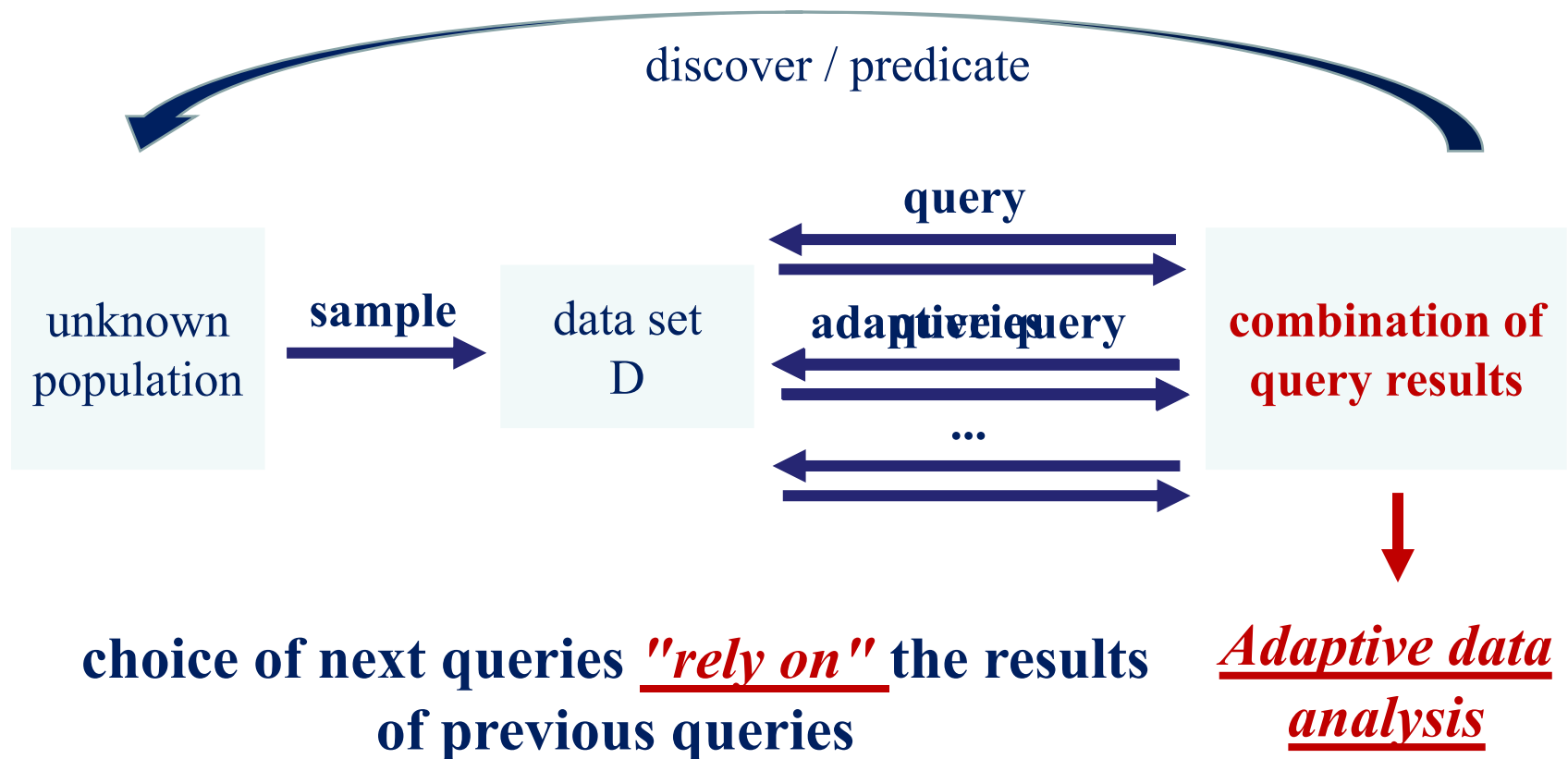
The visiting times of each program location during the program execution

PART I :

Program Analysis for Adaptive Data Analysis

- Introduction:
 - Adaptivity
 - Motivation
- QWhile Language
- Adaptivity Definition
- Adaptivity Estimation Algorithm

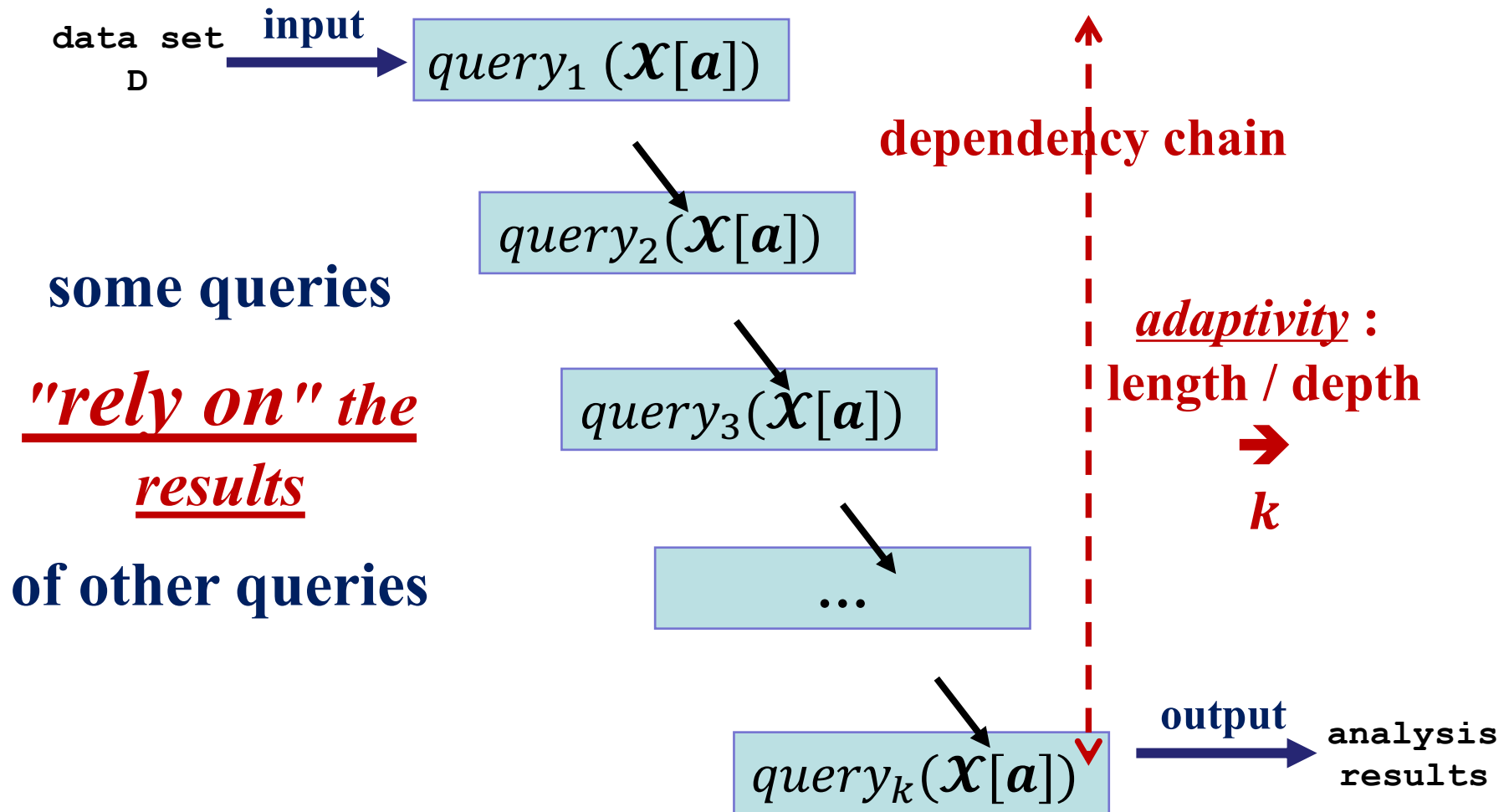
Introduction – adaptive data analysis



example program

```
multiRounds(k) :=  
  [a ← 0] 0;  
  [j ← k] 1;  
  while [(j > 0)] 2 do (  
    [a ← query( $\mathcal{X}[a]$ )] 3;  
    [j ← j - 1] 4  
  )
```

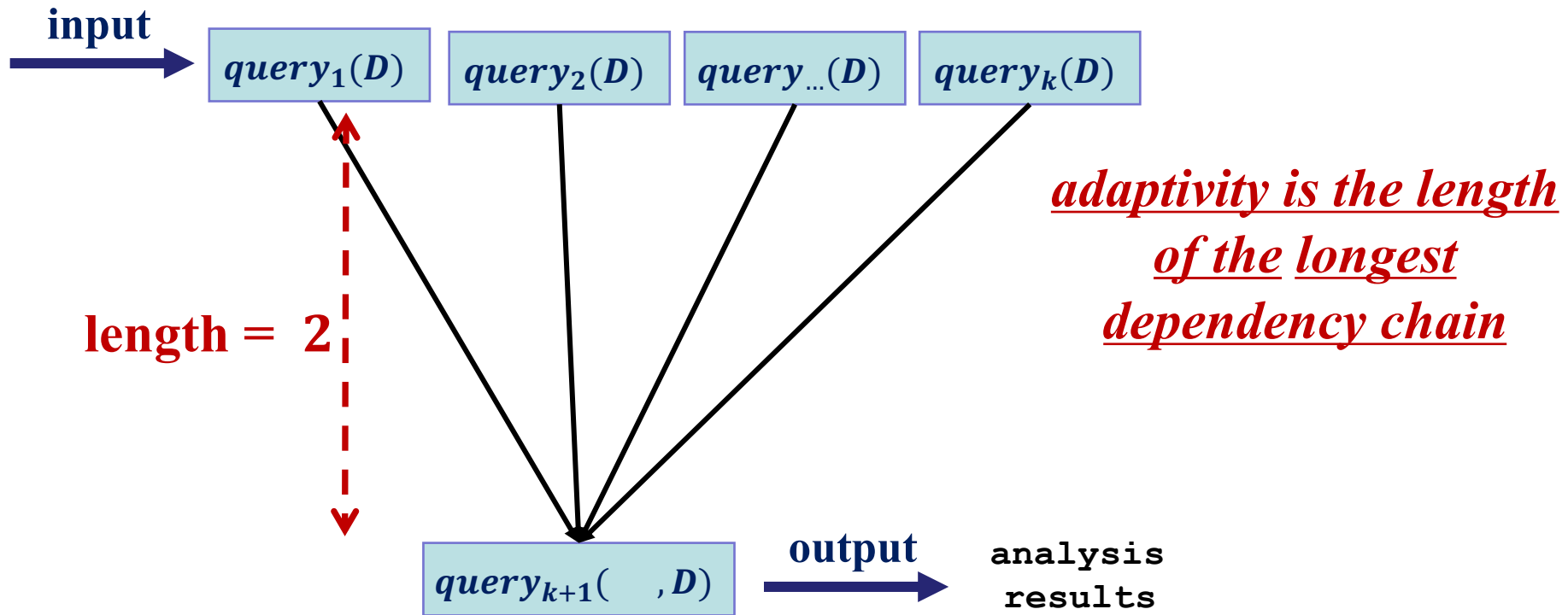
Introduction – adaptivity in adaptive data analysis



QWhile language – example program

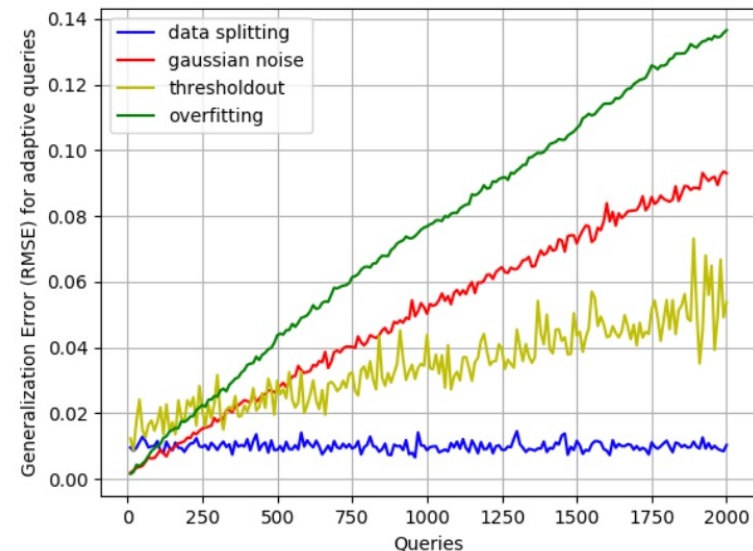
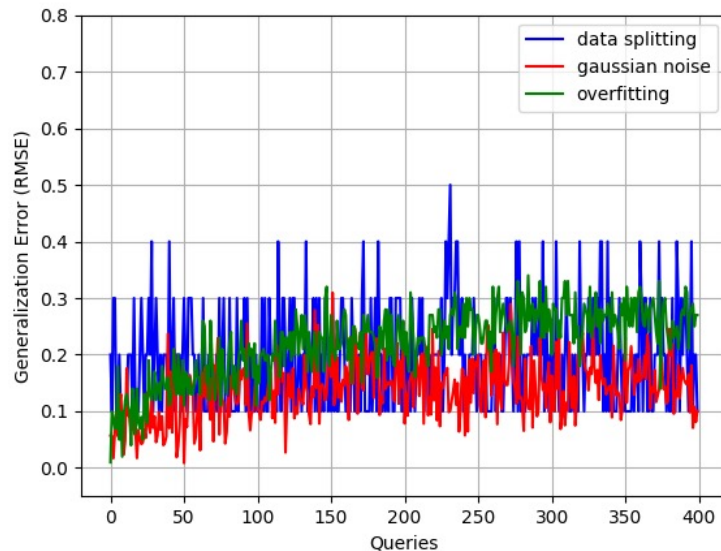
```
twoRounds(k) :=  
  [a ← 0] 0;  
  [j ← k] 1;  
  while [(j > 0)] 2 do (  
    [x ← query( $\mathcal{X}[j]$ )] 3;  
    [j ← j - 1] 4;  
    [a ← a + x] 5);  
  [l ← query( $\mathcal{X}[k] * a$ )] 6
```

Introduction – adaptivity in adaptive data analysis



the "adaptivity" in a data analysis **propagates** the error

Introduction – motivation



use "*adaptivity*" to choose different mechanisms,
to reduce the generalization error

Introduction – research challenges

- Challenge – I : program model for implementing an adaptive data analysis algorithm
- Challenge – II : how to define this quantitative-reachability property
- Challenge – III : how to estimate this quantitative-reachability property accurately

Contributions

- I : QWhile language
- II : Formal adaptivity definition
- III : Program analysis algorithm for adaptivity
- IV: Soundness proof
- V: Prototype implementation

Adaptivity Analysis Framework

QWhile language

Expression	$e ::= v \mid a \mid b \mid [e, \dots, e]$
Value	$v ::= n \mid \text{true} \mid \text{false} \mid [] \mid [v, \dots, v]$
Query Expression	$\psi ::= \alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$
Query Value	$\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$
Labeled Command	$c ::= [x \leftarrow e]^l \mid [x \leftarrow \text{query}(\psi)]^l \mid \text{while } [b]^l \text{ do } c$ $\mid c; c \mid \text{if } ([b]^l, c, c) \mid [\text{skip}]^l$

QWhile language – query expression

Query Expression $\psi ::= \alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$

Query Value $\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$

- $\chi[n]$: access the values at *n-th column* of a *hidden* database
- $\text{query}(\chi[n])$: the *average* value of $\chi[n]$ over each row of the database

$$\text{query}(\chi[3] + 5) = \frac{1}{n} \sum_{i=0}^n \chi_i[3] + 5$$

QWhile language – trace-based semantics

- *Query command evaluation*

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{query}$$

- 1. *Normalize the query expression*: $\langle \tau, \psi \rangle \Downarrow_q \alpha$.

The query expression ψ is first simplified to its normal form α

$$\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$$

$$\frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, a \rangle \Downarrow_q n} \quad \frac{\langle \tau, \psi_1 \rangle \Downarrow_q \alpha_1 \quad \langle \tau, \psi_2 \rangle \Downarrow_q \alpha_2}{\langle \tau, \psi_1 \oplus_a \psi_2 \rangle \Downarrow_q \alpha_1 \oplus_a \alpha_2} \quad \frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, \chi[a] \rangle \Downarrow_q \chi[n]} \quad \frac{}{\langle \tau, \alpha \rangle \Downarrow_q \alpha}$$

QWhile language – trace-based semantics

- Query command evaluation

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{query}$$

- 2. Get the result from hidden database: $\text{query}(\alpha) = v$

Expected value of the function $\lambda \chi . \alpha$ applied to each row of the hidden database:

$$\text{query}(\chi[3] + 5) = \frac{1}{n} \sum_{i=0}^n \chi_i[3] + 5$$

$$\text{Abstract procedure } \text{query}(\alpha) = v$$

QWhile language – trace-based semantics

- *Query command evaluation*

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{query}$$

- 3. Generate the event for query assignment : $\epsilon = (x, l, v, \alpha)$

x : the variable name

l : the label for this query command

v : the query result computed over unknown database

α : the normalized the query expression

Adaptivity Formalization

- ➔ "Reachability" sub-property of Adaptivity
- ➔ "Quantitative" sub-property of Adaptivity
- ➔ "Dependency Graph" carrying the two sub-properties
- ➔ "Adaptivity" as a quantitative-reachability property

→ "Reachability" as **dependency relation**^[1]

- Variable may-dependency:

$$\text{DEP}(x^i, y^j, c) \in \{\text{true}, \text{false}\}$$

```

twoRounds (k)  :=
  [a ← 0] 0;
  [j ← k] 1;
  while [(j > 0)] 2 do (
    [x ← query(X[j])] 3;
    [j ← j - 1] 4;
    [a ← a + x] 5);
  [1 ← query(X[k] * a)] 6

```

- $\text{DEP}(x^3, a^5, c), \text{DEP}(a^5, l^6, c)$

[1] Patrick Cousot. 2019. Abstract Semantic Dependency. SAS'19

→ "Quantitative" as the weight of a labeled variable^[1,2]

- Weight of each labeled variable x^l in program c :

$$w: \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$$

if $\rho(\tau_0, k) = 10$

```

twoRounds(k) :=
  [a ← 0]0; → 1
  [j ← k]1; → 1
  while [(j > 0)]2 do (
    [x ← query( $\mathcal{X}[j]$ )]3; →
    [j ← i - 1]4; → 10
    [a ← a + x]5; →
    [1 ← query( $\mathcal{X}[k] * a$ )]6 → 1

```

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Example – quantitative as weight of labeled variables

```

twoRounds(k)  :=
  [a ← 0] 0;
  [j ← k] 1;
  while [(j > 0)] 2 do (
    [x ← query( $\mathcal{X}[j]$ )] 3;
    [j ← j - 1] 4;
    [a ← a + x] 5);
  [1 ← query( $\mathcal{X}[k] * a$ )] 6

```

- for a^0 : $w = \lambda \tau_0.1$,
- for x^3 : $w = \lambda \tau_0.\max(0, \rho(\tau_0, k))$

➔ New "structure" as semantics-based dependency graph

$$G(c) = (V(c), E(c), W(c), Q(c))$$

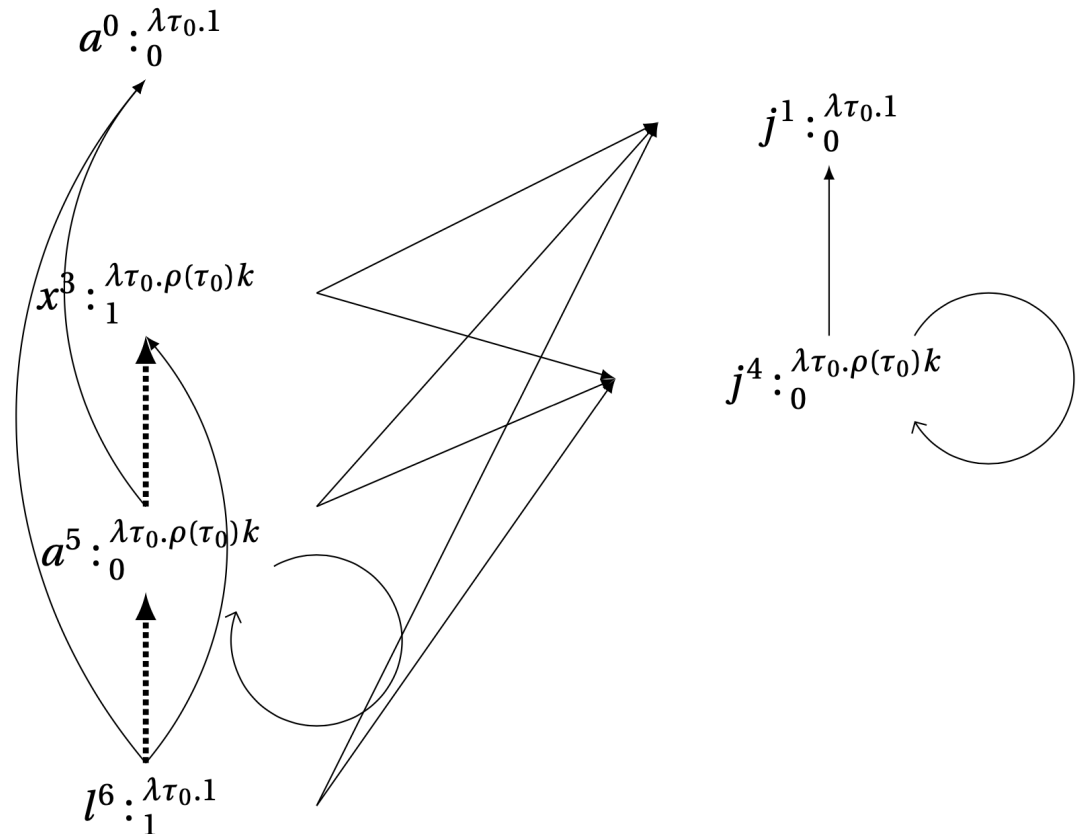
- Vertex $V(c)$: all the labeled variables in c ;
- Edge $E(c)$: *variable may-dependency* ;
- Weight $W(c)$: *weight* of each labeled variable x^l in c ;
- Query Annotation $Q(c)$: *query request*.

encoding the two sub-properties


```

twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do
(
[x ← query( $\mathcal{X}[j]$ )]3;
[j ← j - 1]4;
[a ← a + x]5);
[l ← query( $\mathcal{X}[k] * a$ )]6

```



→ Adaptivity as a "quantitative-reachability" hyper-property

$$\text{Adaptivity } A(c) : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$$

$$A(c) = \lambda \tau_0. \max\{\text{length of the } \textit{all the finite walks} \}$$

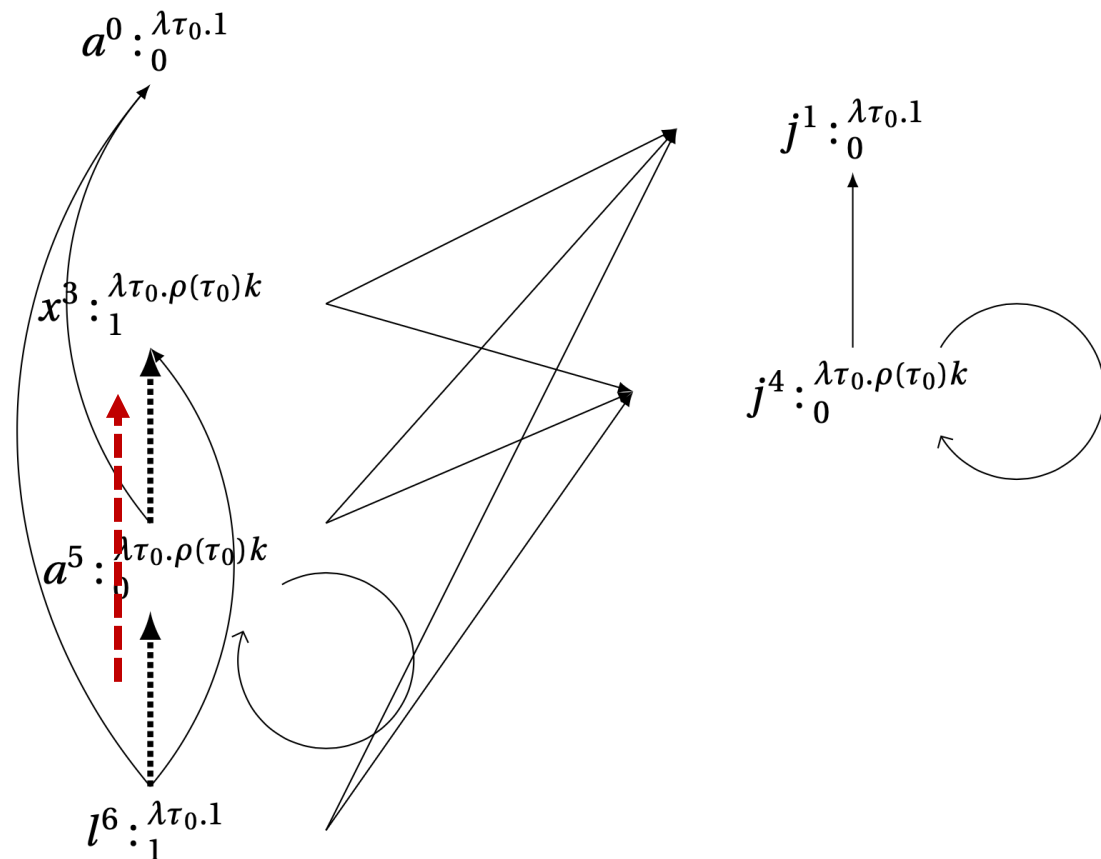
- finite walk : visiting times bounded by weight
- length : count only the query request

Example – adaptivity as the length of the longest finite walk

```

twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do
(
[x ← query(X[j])]3;
[j ← j - 1]4;
[a ← a + x]5);
[l ← query(X[k] * a)]6

```



Program Analysis Algorithm for Adaptivity

- ➔ "Reachability" sub-property:
data dependency estimation by data & control-flow analysis
- ➔ "Quantitative" sub-property :
weight estimation by reachability-bound analysis
- ➔ Estimate "structure"
estimated dependency graph
- ➔ "Adaptivity"
compute upper bound by longest walk estimation algorithm

→ "Reachability": data-dependency analysis

data & control-flow and reaching definition analysis^[1]

$$\text{flowsTo}(x^i, y^j, c) \in \{\text{true}, \text{false}\}$$

Soundness

$$\begin{aligned} \forall x^i, y^j \in \text{LV}(c) . \text{DEP}_{\text{var}}(x^i, y^j, c) \\ \implies \left(\exists n \in \mathbb{N}, z_1^{r_1}, \dots, z_n^{r_n} \in \text{LV}(c) . n \geq 0 \right. \\ \left. \wedge \text{flowsTo}(x^i, z_1^{r_1}, c) \wedge \dots \wedge \text{flowsTo}(z_n^{r_n}, y^j, c) \right) \end{aligned}$$

[1] Nielson F., H.R. Nielson; , C. Hankin (2005). Principles of Program Analysis. Springer. [ISBN 3-540-65410-0](#).

→ "Quantitative" analysis[1]

$\hat{w} \in \text{Expr}(\mathbb{N} \cup \text{Input} \cup \{\infty\})$

```

twoRounds(k) :=
[a ← 0]0; → 1
[j ← k]1; →
while [(j > 0)]2 do (
    [x ← query(X[j])]3; →
    [j ← i - 1]4; → k
    [a ← a + x]5); →
[l ← query(X[k] * a)]6 → 1

```

Soundness:

$\forall c \in \mathcal{C}, x^l \in \text{LV}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, v \in \mathbb{N}, l', (x^l, w) \in W_{\text{trace}}(c) .$

$(x^l, \hat{w}) \in W_{\text{est}}(c) \wedge (\langle \tau_0, \hat{w} \rangle \Downarrow_e v \implies w(\tau_0) \leq v)$

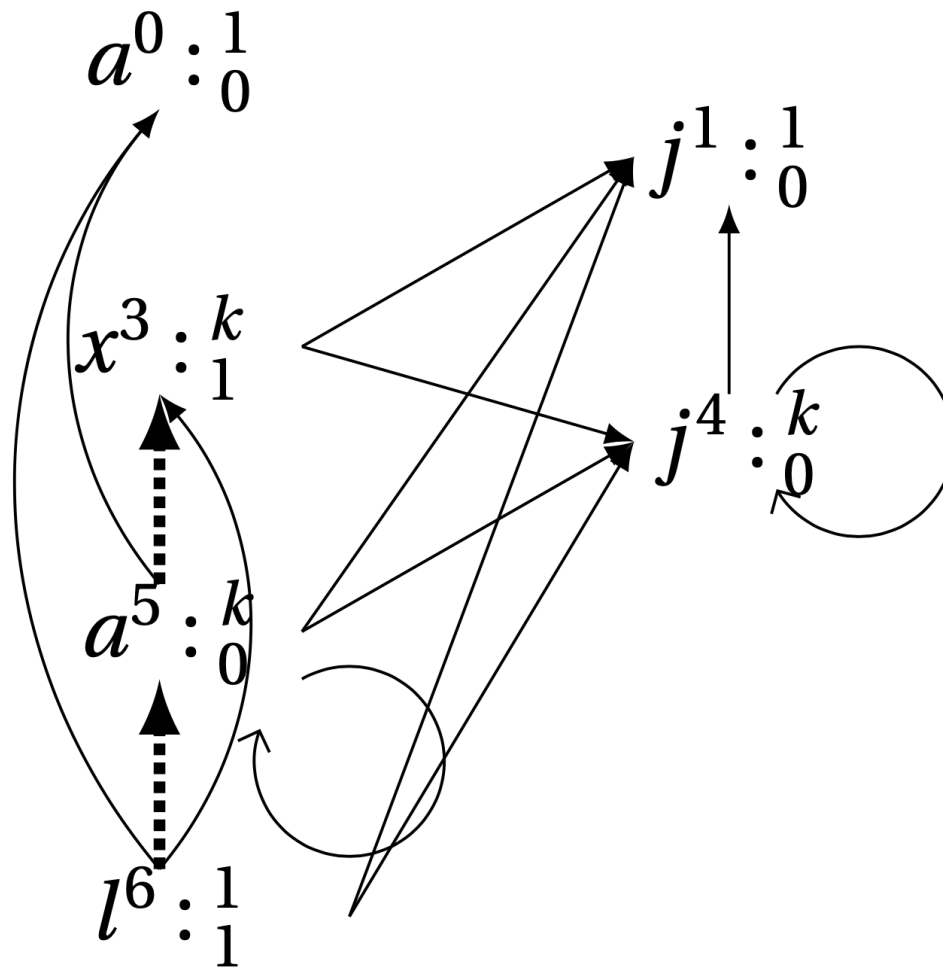
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

➔ Estimate *semantics-based dependency graph*

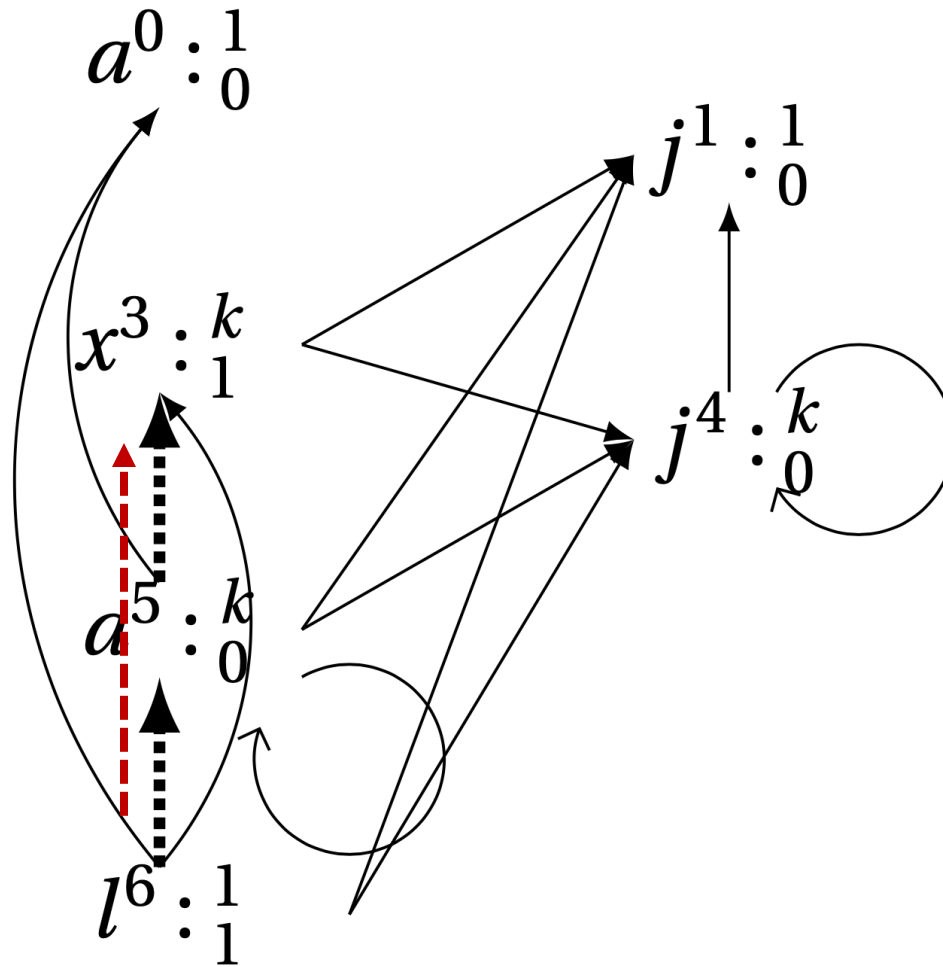
$$\widehat{G}(c) = (V(c), \widehat{E}(c), \widehat{W}(c), Q(c))$$

- Vertex $V(c)$: labeled variables in c
- Edge $\widehat{E}(c)$: *Estimated data-dependency relation*, $\text{flowsTo}(x^i, y^j, c)$
- Weight $\widehat{W}(c)$: *Estimated weight* for each labeled variable x^l in c ;
- Query Annotation $Q(c)$: labeled variables assigned by *query requests*.

Example – *the estimated dependency graph*



→ Longest walk search algorithm



→ Implementation and empirical evaluation

Program c	adaptivity	AdaptFun		L.O.C	running time (second)		
		Aest	query#		graph	weight	AdaptBD
twoRound(k)	2	2	$k + 1$	8	0.0005	0.0017	0.0003
mR(k)	k	k	k	10	0.0012	0.0017	0.0002
lRGD(k, r)	k	k	$2k$	10	0.0015	0.0072	0.0002
mROdd(k)	$1 + k$	$2 + \max(1, 2k)$	$1 + 3k$	10	0.0015	0.0061	0.0002
mRSingle(k)	2	$1 + \max(1, k)$	$1 + k$	9	0.0011	0.0075	0.0002
ifCD()	3	3	3	5	0.0005	0.0003	0.0001
loop(k)	$1 + k/2$	$1 + \max(1, k/2)$	$1 + k/2$	7	0.0021	0.0015	0.0001
loopRV(k)	$1 + 2k$	$1 + 2k$	$2 + 3k$	9	0.0016	0.0056	0.0001
loopVCD(k)	$1 + 2Q_m$	$Q_m + \max(1, 2Q_m)$	$2 + 2Q_m$	6	0.0016	0.0007	0.0001
loopMPVCD(k)	$2 + Q_m$	$2 + Q_m$	$2 + 2Q_m$	9	0.0017	0.0043	0.0001
loop2VD(k)	$2 + k^2$	$3 + k^2$	$1 + k + k^2$	10	0.0018	0.0126	0.0001
loop2RV(k)	$1 + k + k^2$	$2 + k + k^2$	$2 + k + k^2$	10	0.0017	0.0186	0.0001
loop2MV(k)	$1 + 2k$	$1 + \max(1, 2k)$	$1 + k + k^2$	10	0.0016	0.0071	0.0001
loop2MPRV(k)	$1 + k + k^2$	$3 + k + k^2$	$2 + 2k + k^2$	10	0.019	0.0999	0.0002
loopM(k)	$1 + k$	$2 + \max(1, 2k)$	$1 + 3k$	9	0.0017	0.0062	0.0001
loopM2(k)	$1 + k$	$2 + k$	$1 + 3k$	9	0.0017	0.0062	0.0001
loop2RC(k)	$1 + 3k$	$1 + 3k$	$1 + 3k + k^2$	11	0.019	0.2669	0.0007
mR(k, N)	k	k	k	27	0.0026	85.9017	0.0004
mRC(k)	$2k$	$2k$	$2k$	46	0.0036	5104	0.0013
seqC(k)	12	12	326	502	0.0426	1.2743	0.0223
tRC(k)	2	*	*	42	0.0026	*	0.0005
jumbo(k)	$\max(20, 8 + k^2)$	*	*	71	0.0035	*	0.0085
big(k)	$22 + k + k * k$	*	*	214	0.0175	*	0.002

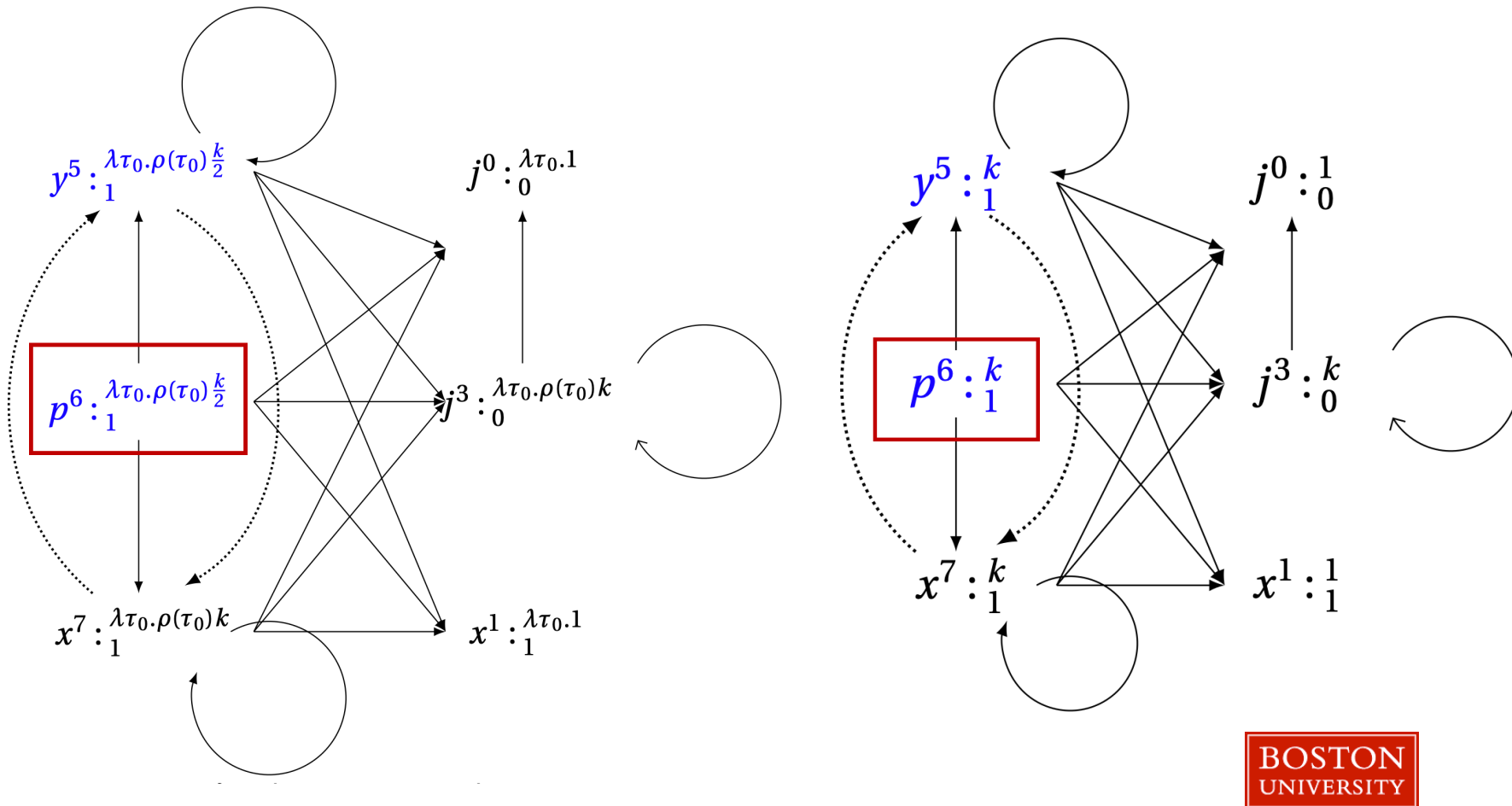
Limitation – imprecision of the program analysis algorithm

- The over-approximation in multiple paths loop

```
oddRounds(k) :=  
  [j ← 0] 0;  
  [x ← query( $\mathcal{X}$ [0])] 1;  
  while [(j > 0)] 2 do  
  (  
    [j ← j - 1] 3;  
    if ([j%2 == 0] 4,  
        [y ← x] 4; [p ← x] 5),  
        [x ← query( $\mathcal{X}$ [y])] 6  
    )  
  )
```

Limitation – imprecision of the program analysis algorithm

- The over-approximation in multiple paths loop

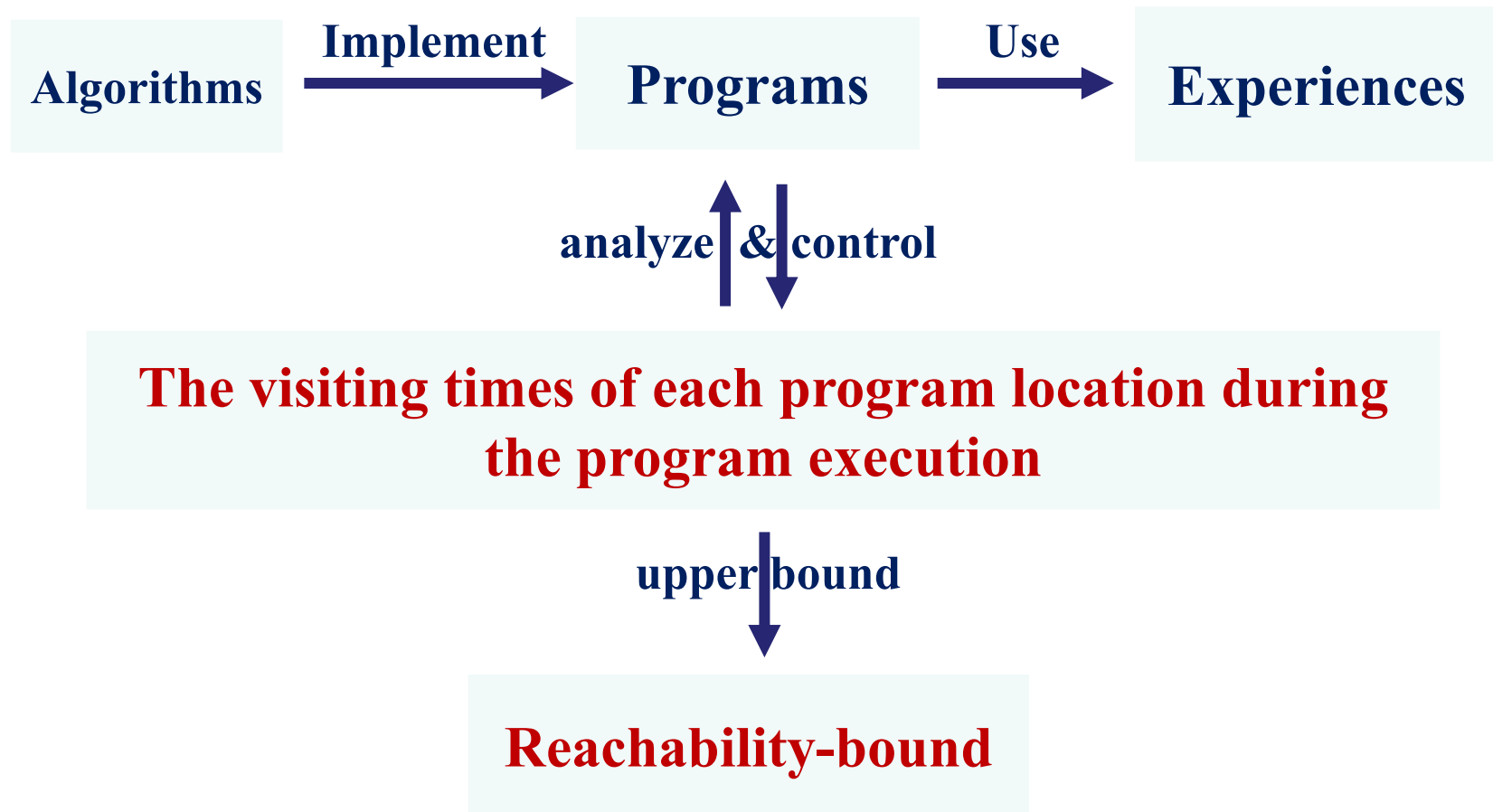


PART II :

Path-sensitive Reachability-bound Analysis

- Introduction:
 - Background
 - Motivation
- Language Model
- The Reachability-bound
- Path-sensitive Reachability-bound Algorithm

Introduction – background



Introduction – motivation

the number of times a given program location inside a procedure is visited during the program execution.

```

oddRounds(k) :=
[j ← 0]0;
[x ← query( $\mathcal{X}$ [0])]1;
while [(j > 0)]2 do
(
[j ← j - 1]3;
if ([j%2 == 0]4,
  [y ← x]4; [p ← x]5),
  [x ← query( $\mathcal{X}$ [y])]6 )
)

```

simplify

```

twoPaths(k) :=
[i ← k]0, → 1
while [i > 0]1 do → k/2+1
( if ([i%2==0]2, → k/2
  then [i ← i - 1]3, → k/4
  else [i ← i - 3]4) → k/4
)

```

Introduction – technique challenges in the path-sensitivity state-of-art^[3]

```

twoPaths(k) :=
  [i ← k]0 , → ?
  while [i > 0]1 do → k/2
    (if([i%2==0]2, → k/2
      then [i ← i - 1]3, → k/2 > k/4
      else [i ← i - 3]4) → k/2 > k/4

```

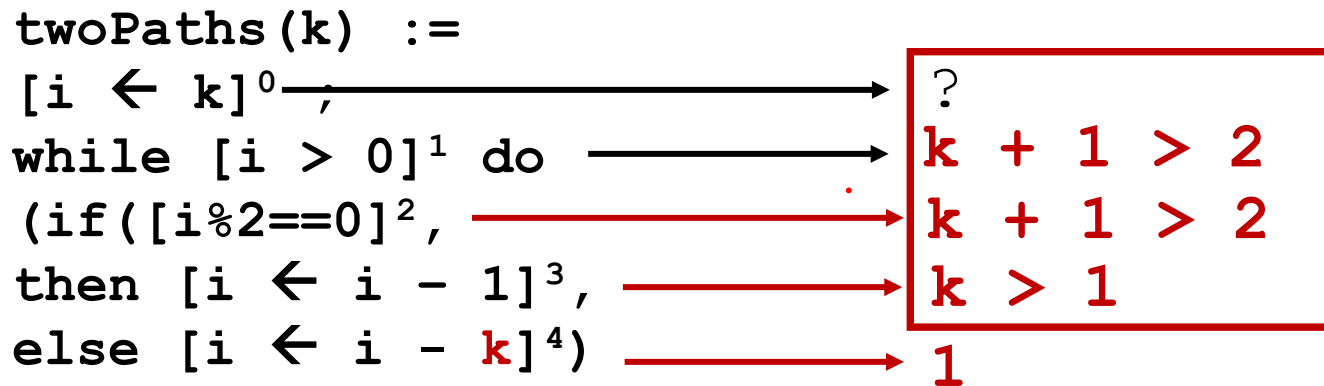
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in the path-sensitivity

state-of-art^[1, 2, 3]



[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in nested loops

multiple loop paths and nested loops

```

loop2 (n, m) :=
  [i ← n]0 ;
  while [i > 0]1 do
    (if ([i%2==0]2,
      then
        [k ← i - m]3;  $\longrightarrow \frac{m}{4}$ 
        while [i > 0]4 do ([k ← i - m]5) ;  $\longrightarrow n - m$ 
        [i ← k + m]6;
        [i ← i - 1]7,
      else
        [i ← i - 3]8
    )
  )

```

Introduction – technique challenges in nested loops

state-of-art^[1,2,3]

```

loop2 (n, m) :=
  [i ← n]0 ;
  while [i > 0]1 do
    (if ([i%2==0]2,
      then
        [k ← i - m]3;
        while [i > 0]4 do ([k ← i - m]5) ;
        [i ← k + m]6;
        [i ← i - 1]7,
      else
        [i ← i - 3]8
    )
  )

```

$$\begin{array}{c}
 \xrightarrow{\frac{m}{2} > \frac{m}{4}} \\
 \downarrow \\
 \frac{(n - m) * m}{2} > n - m
 \end{array}$$

Introduction – main contribution

- New algorithm
 - compute reachability-bound on each location
 - compute different bounds for locations on different loop paths
- New quantities:
 - Loop reachability-bound
 - Path reachability-bound
- Implementation and evaluations

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Language model – expression evaluation

$$\llbracket \cdot \rrbracket : \mathcal{A} \cup \mathcal{B} \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{V}$$

 $\llbracket \cdot \rrbracket : \text{Arithmetic Expression} \rightarrow \text{Trace} \rightarrow \text{Arithmetic Value}$

$$\begin{array}{c}
 \frac{}{\llbracket n \rrbracket(\tau) = n} \qquad \frac{\rho(\tau)x = v}{\llbracket x \rrbracket(\tau) = v} \qquad \frac{\llbracket a_1 \rrbracket(\tau) = v_1 \quad \llbracket a_2 \rrbracket(\tau) = v_2 \quad v_1 \oplus_a v_2 = v}{\llbracket a_1 \oplus_a a_2 \rrbracket(\tau) = v} \\
 \\
 \frac{\llbracket a \rrbracket(\tau) = v' \quad \log v' = v}{\llbracket \log a \rrbracket(\tau) = v} \qquad \frac{\llbracket a \rrbracket(\tau) = v' \quad \text{sign } v' = v}{\llbracket \text{sign } a \rrbracket(\tau) = v}
 \end{array}$$

The estimated reachability-bound **psRB** $\in \mathcal{A} \cup \mathcal{B}$

The reachability-bound – definition^[1]

label counter $\text{cnt} : \mathcal{T}^{+\infty} \rightarrow \mathcal{L} \rightarrow \mathbb{N} \cup \{\perp\}$

return $\perp$ for any label if the trace is ∞

Definition 3 (Reachability-bound) *For a program c and a location $l \in \mathbf{L}(c)$ in c , a function $f(c, l) : \mathcal{T}_0^+(c) \rightarrow (\mathbb{N} \cup \{\infty\})$ is a reachability-bound for l if and only if*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), c' \in \mathcal{C}, l, l' \in \mathbf{L}(c), \tau \in \mathcal{T}^{+\infty} . \\ \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\text{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \implies f(c, l)(\tau_0) \geq \text{cnt}(\tau, l)$$

[1] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Path-sensitive reachability-bound algorithm

1. Generate An Abstract Transition Graph
2. ***Refine*** the multipaths of loops in Program
3. Estimate the ***ranking functions*** and bound invariant
4. Estimate the ***path reachability-bound*** for every ***path***
5. Estimate the ***loop reachability-bound*** w.r.t. ***path*** and ***outer loops***
6. Estimate the ***reachability-bound*** for every ***program point***

→ Abstract transition graph:

$$\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$$

- **Vertex**: $\text{absV}(c) = L(c) \cup \{\text{ex}\}$
- **Edges**: $\hat{e} = (l, dc, l')$

Constraints dc:

- Difference constraints
- Boolean expressions

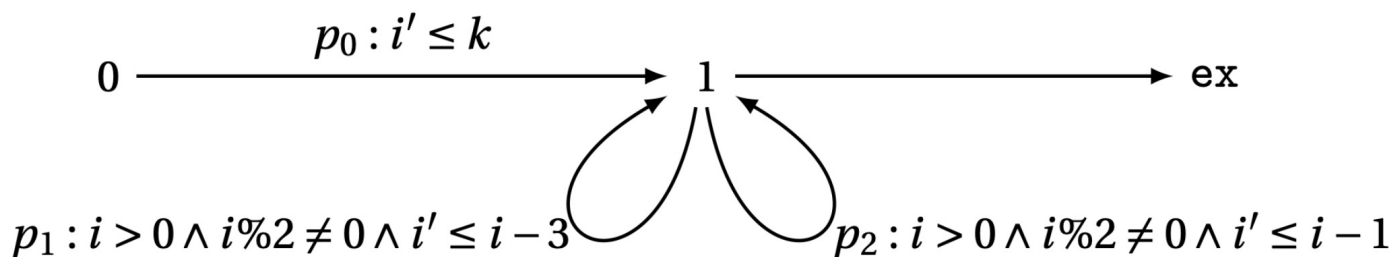
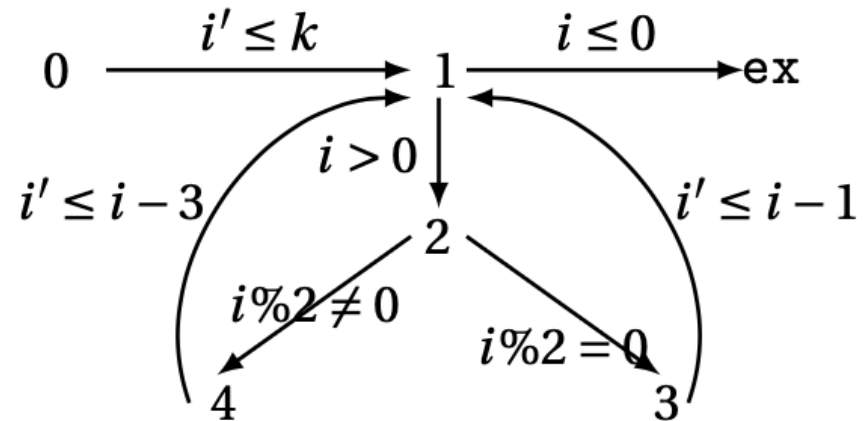
contains every program point and transition

Example - abstract transition graph

```

twoPaths(k) :=
  [i ← k]0 ;
  while [i > 0]1 do
    (if([i%2==0]2,
      then [i ← i - 1]3,
      else [i ← i - 3]4)

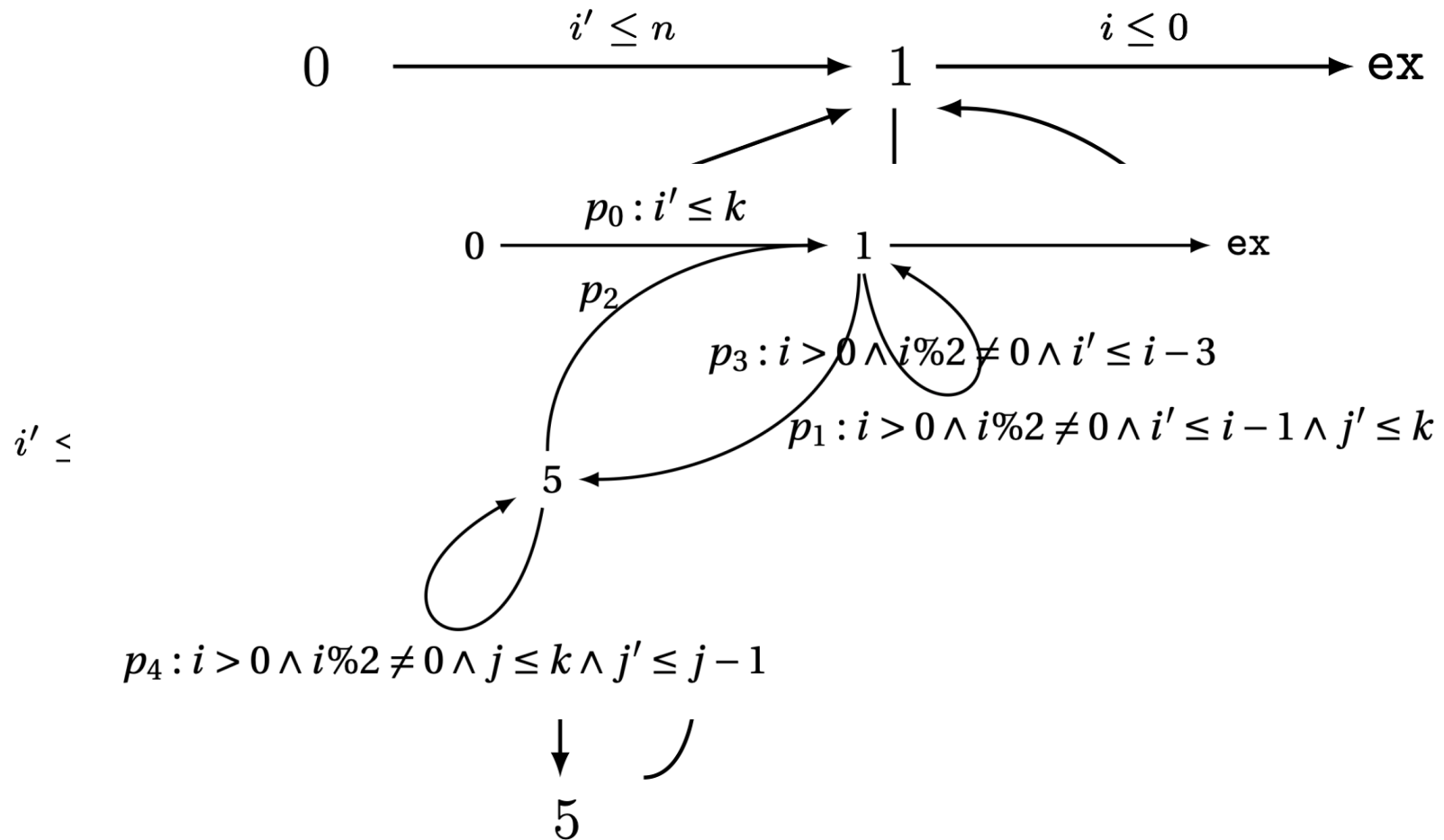
```



Example - abstract transition graph

```
loop2(n, m) :=  
  [i ← n]0 ;  
  while [i > 0]1 do  
    (if([i%2==0]2,  
      then  
        [k ← i - m]3;  
        while [i > 0]4 do ([k ← i - m]5;  
          [i ← k + m]6;  
          [i ← i - 1]7,  
        else  
          [i ← i - 3]8  
      )  
    )
```

Example - abstract transition graph



→ Abstract transition graph – Path

$$\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$$

■ Path:

$$p \in \mathcal{PH}(\text{absG}(c))$$

■ path

$1 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow \text{ex}$

■ cyclic path

$1 \rightarrow 2 \rightarrow 4 \rightarrow 1$

■ simple path

$0 \rightarrow 1, 1 \rightarrow \text{ex}, 1 \rightarrow 2 \rightarrow 3 \rightarrow 1, 0 \rightarrow 1 \rightarrow \text{ex}$

→ Abstract transition graph – *Simple transition path*

$$\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$$

■ *Simple transition path:*

$$\text{tp} \in \mathcal{PH}(\text{absG}(c))$$

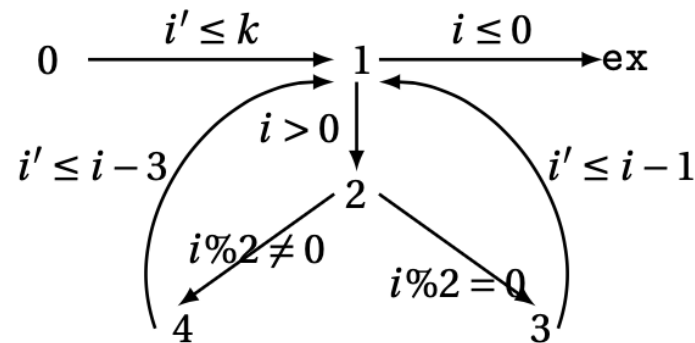
simple and start- or end- point **has to** be either

- a loop header
- or program entry
- or program exit.

and no loop header except the start- or end- points.

loop free and interleaving free

Example - simple transition path



- Simple transition path

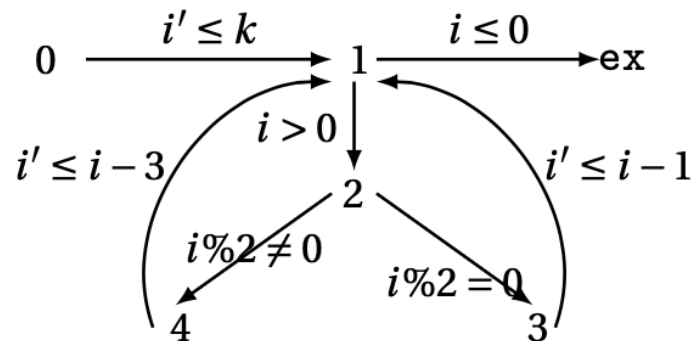
$0 \rightarrow 1, 1 \rightarrow \text{ex}, 1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

- Path that is simple but not a simple transition path

$0 \rightarrow 1 \rightarrow \text{ex}$

→ Program refinement – *new loop paths*

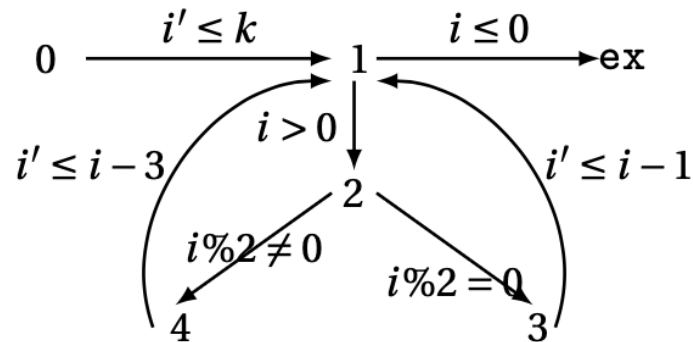
- Rewrite and compute new loop paths



- New loop paths
 - $rp1 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1 = tp1; tp2$
 - $rp2 = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1 = tp2; tp1$
- The refined program, *rp*
 - $rp = tp1; \text{choose}\{\text{repeat}(rp1), \text{repeat}(rp2)\}; tp3$

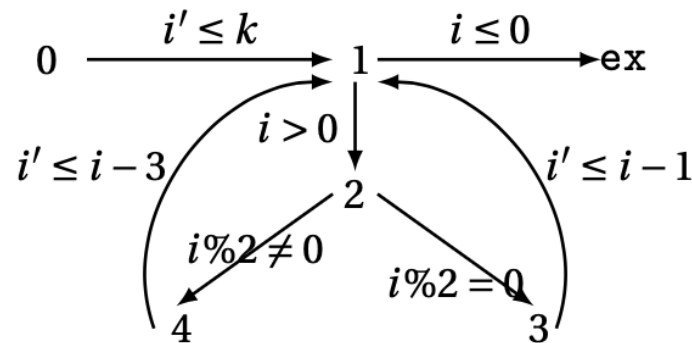
Iteration times of *new loop paths*.

- $\text{rp1} = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1$
- $\text{rp2} = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

$$\frac{k}{4}$$


→ Loop bound for *new loop paths*

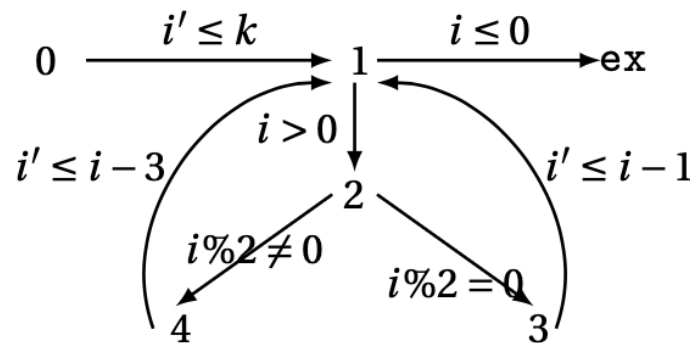
$$\underline{BD(rp, c)} \in Expr(\mathbb{N} \cup Input \cup \{\infty\})$$



- $BD(1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1) \Rightarrow \frac{k-0}{4}$
- $BD(1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1) \Rightarrow \frac{k-0}{4}$

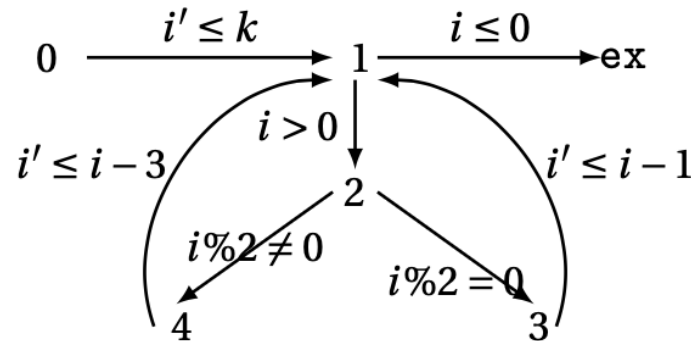
Iteration times of *simple transition paths*.

- $\text{tp1} = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1,$
- $\text{tp2} = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1$

$$\frac{k}{4}$$


➔ **Path reachability-bounds** – locally

$\text{pathLocRB}(rp, tp) \in \text{Expr}(\mathbb{N} \cup \text{Input} \cup \{\infty\})$



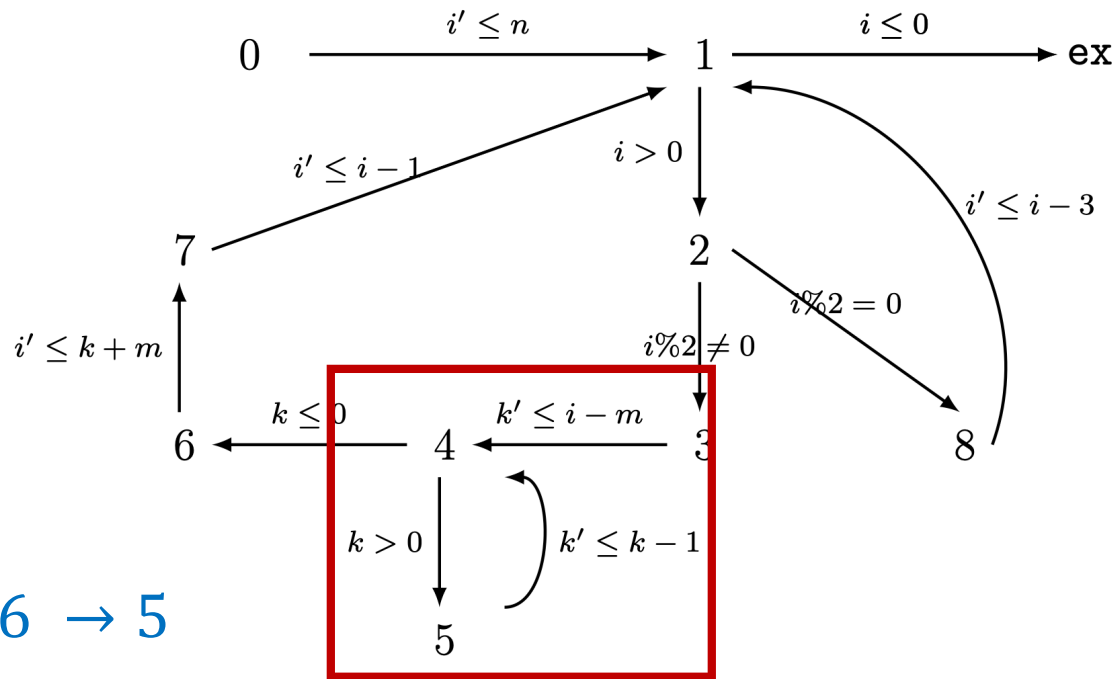
$\text{pathlocRB}(rp, tp1)$

$$= \max \left\{ \text{pathlocRB}(rp1, tp1), \right. \\ \left. \text{pathlocRB}(rp2, tp1) \right\}$$

$$= \max \left\{ \text{pathlocRB}(tp1; tp2, tp1) * \text{BD}(rp1, c), \right. \\ \left. \text{pathlocRB}(tp1; tp2, tp1) * \text{BD}(rp1, c) \right\}$$

$$= \max \left\{ 1 * \frac{k}{4}, 1 * \frac{k}{4} \right\} = \frac{k}{4}$$

Not precise in case with nested loops.



$tp = 5 \rightarrow 6 \rightarrow 5$

$rp1 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4; \text{repeat}(4 \rightarrow 5 \rightarrow 4); 4 \rightarrow 6 \rightarrow 7 \rightarrow 1$

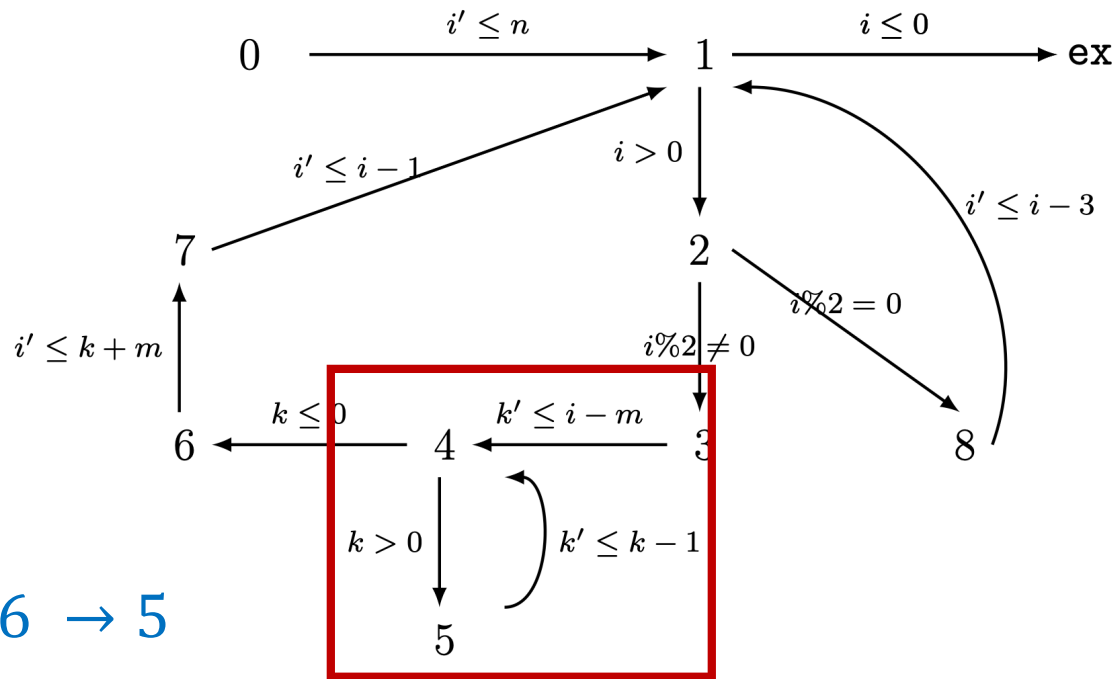
$$BD(rp1) * BD(tp) = \frac{(n-m)*m}{4} > n - m$$

→ Loop reachability-bound

$$\text{lpRB}(rp', tp) \in \text{Expr}(\mathbb{N} \cup \text{Input} \cup \{\infty\})$$

The upper bound on iteration numbers of the outer loop, such that, during these iterations, the inner loop is entered.

Not precise in case with nested loops.



$tp = 5 \rightarrow 6 \rightarrow 5$

$rp1 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4; \text{repeat}(4 \rightarrow 5 \rightarrow 4); 4 \rightarrow 6 \rightarrow 7 \rightarrow 1$

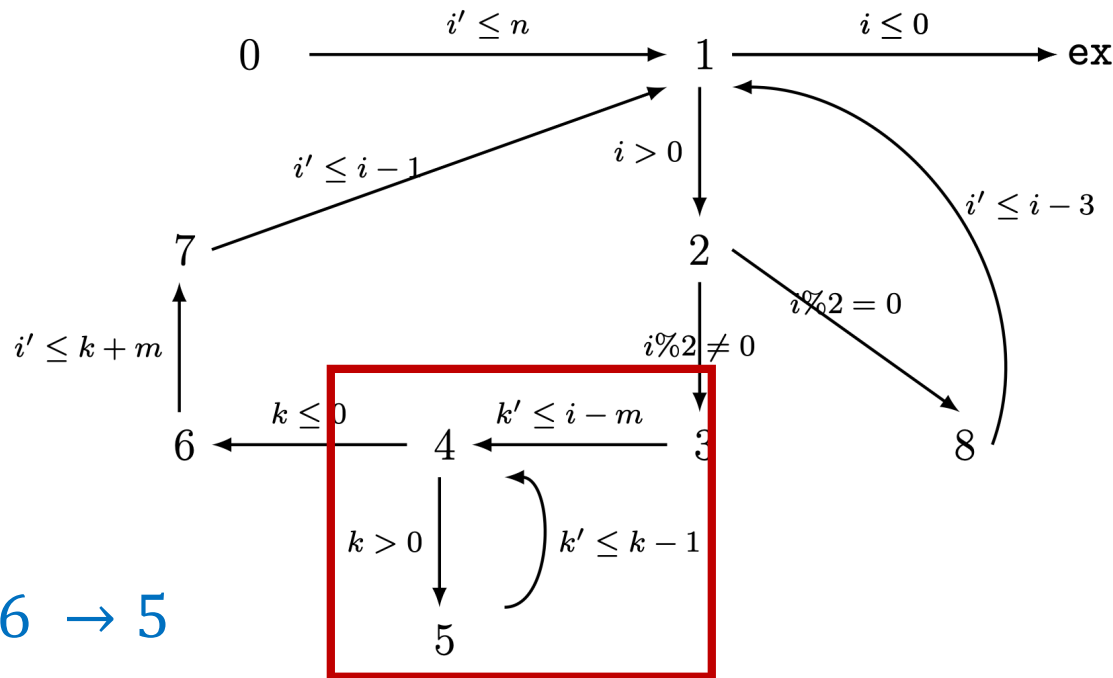
$\text{loopRB}(rp1, tp) = 1$

➔ *Path reachability-bound* – globally

$$\text{pathRB}(tp, c) \in \text{Expr}(\mathbb{N} \cup \text{Input} \cup \{\infty\})$$

the upper bound on the execution times of a simple transition path, tp when executing the program.

Not precise in case with nested loops.



$tp = 5 \rightarrow 6 \rightarrow 5$

$rp1 = 1 \rightarrow 2 \rightarrow 3 \rightarrow 4; \text{repeat}(4 \rightarrow 5 \rightarrow 4); 4 \rightarrow 6 \rightarrow 7 \rightarrow 1$

$\text{pathRB}(rp1, tp) = 1 * (n - m) = n - m$

→ Reachability-bound for every location

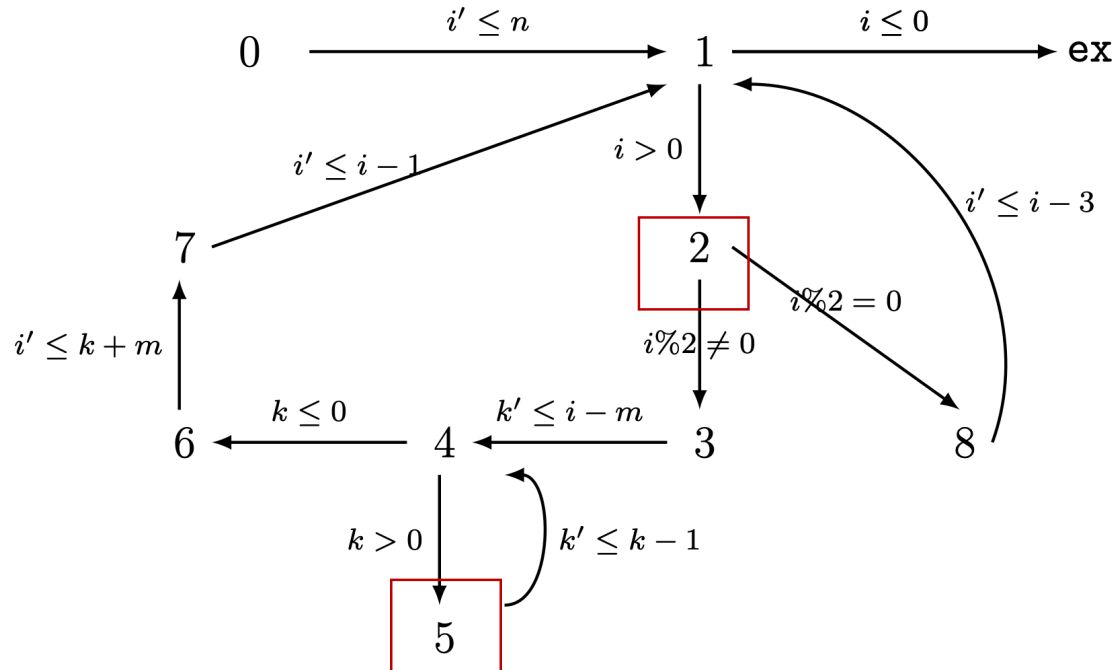
$$\text{psRB}(tp, c) \in \text{Expr}(N \cup \text{Input} \cup \{\infty\})$$

$$\text{psRB}(l, c) = \sum \{\text{pathRB}(rp, tp, c) \mid tp \in rp \wedge l \in L_{\text{path}}(tp)\}$$

Soundness:

$\llbracket \text{psRB}(l, c) \rrbracket$ is a reachability-bound for program c at location l

Example



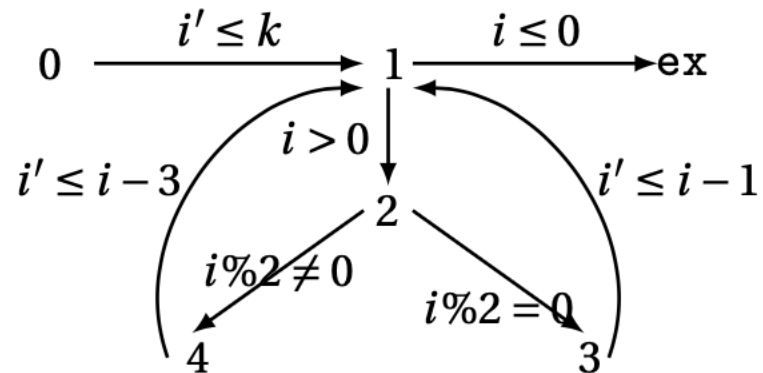
- $\text{psRB}(5) = \mathbf{n - m}$
- $\text{psRB}(2) =$
 $\text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 3)$
 $+ \text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 8 \rightarrow 1) = \mathbf{\frac{m}{2}}$

Example

```

twoPaths(k) :=
  [i ← k]0
  while [i > 0]1 do
    (if ([i%2==0]2,
      then [i ← i - 1]3,
      else [i ← i - 3]4)

```



- $\text{psRB}(4) = \text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 4 \rightarrow 1) = \frac{m}{4}$
- $\text{psRB}(2) = \text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 4 \rightarrow 1) + \text{pathRB}(\text{rp}, 1 \rightarrow 2 \rightarrow 3 \rightarrow 1) = \frac{m}{2}$

➔ *Implementation and empirical evaluation*

Benchmarks: 270 from four benchmark sets

- 20 self-adopted programs
- 75.c programs from 7 benchmarks:

ABC: 15, C4B: 15, KoAT: 3, Loopus: 10, Rank10: 3, SPEED: 27, WTC: 37

- 37 challenging.c programs from the **LoopusJAR** [1]
- 37.java programs from **Tianhan**[2]
- 118.c programs from **Icra** [3]

[1] <https://forsyte.at/static/people/sinn/loopusJAR/index.html>

[2] <https://zenodo.org/record/5140586#.Y5pBoC-B1QI>

[3] <https://github.com/icra-team/icra>

Boston University Department of Computer Science

Effectiveness evaluation

Bench- marks	theory worst	(# p)	(# l)	reachability-bound (#1)						worst bound (# p)				
				1	n	n^2	n^3	n^4	∞	psRB	Loopus	Cofloco	SPEED	Lu (2021)
Represent- tives	$O(1)$	4	82	32	0	0	0	0	0	1	2	3	2	1
	$O(n)$	59	373	112	261	0	0	0	0	48	51	45	46	40
	$O(n^2)$	31	414	70	191	153	0	0	0	37	29	34	37	49
	$n \log(n)$	6	81	0	0	0	0	0	0	0	0	0	0	0
	$O(n^3)$	2	91	6	16	13	13	0	0	4	3	2	5	7
	$O(n^4)$	3	36	4	9	9	12	11	0	4	4	3	5	5
Loopus	$O(1)$	1	154	0	0	0	0	0	0	0	1	0	0	0
	$O(n)$	13	176	49	105	0	0	0	0	12	13	14	14	11
	$O(n^2)$	4	64	4	13	15	0	0	0	3	2	5	2	6
	$O(n^3)$	4	193	2	7	10	4	0	0	2	1	2	2	3
	$O(n^4)$	1	21	1	3	6	8	2	0	1	1	1	1	0
Icra	$O(1)$	4	93	41	0	0	0	0	0	1	3	2	2	0
	$\log(n)$	2	16	0	0	0	0	0	0	0	0	0	0	0
	$O(n)$	79	988	267	698	0	0	0	0	78	80	82	78	77
	$O(n^2)$	12	151	26	110	26	0	0	0	18	14	11	16	17
	$O(n^3)$	1	9	1	3	3	2	0	0	1	1	4	2	4
	∞	20	295	50	0	0	0	0	245	9	0	0	0	0
Tianhan	$O(n)$	35	188	68	120	0	0	0	0	35	35	35	35	35
	$O(n^2)$	2	12	4	8	4	0	0	0	2	2	2	2	2

Accuracy evaluation

Bench- marks	theory worst	(# p)	(# l)	overall complexity bound	
				psRB	worst
Representen- tatives	$O(1)$	4	82	32	64
	$O(n)$	59	373	$112 + 261n$	$373n$
	$O(n^2)$	31	414	$70 + 191n + 153n^2$	$414n^2$
	$n \log(n)$	6	81	0	0
	$O(n^3)$	2	91	$6 + 16n + 13n^2 + 13n^3$	$48n^3$
	$O(n^4)$	3	36	$4 + 9n + 9n^2 + 12n^3 + 11n^4$	$45n^4$
Loopus	$O(1)$	1	154	0	-
	$O(n)$	13	176	$49 + 105n$	$162n$
	$O(n^2)$	4	64	$4 + 13n + 15n^2$	$32n^2$
	$O(n^3)$	4	193	$2 + 7n + 10n^2 + 4n^3$	$23n^3$
	$O(n^4)$	1	21	$1 + 3n + 6n^2 + 8n^3 + 2n^4$	$20n^4$
Icra	$O(1)$	4	93	6	64
	$\log(n)$	2	16	0	0
	$O(n)$	79	988	$267 + 698n$	$965n$
	$O(n^2)$	12	151	$26 + 110n + 26n^2$	$162n^2$
	$O(n^3)$	1	9	$1 + 3n + 3n^2 + 2n^3$	$9n^3$
	∞	20	295	50	∞
Tianhan	$O(n)$	35	188	$68 + 120n$	$188n$
	$O(n^2)$	2	12	$4 + 8n + 4n^2$	$16n^2$

Performance evaluation

Benchmarks	# P	M.P.L #	Nested Loop #	Bounded #	L.O.C	Success Rate	Failed	Time Outs	Total Runtime
Representatives	110	53	52	107	954	85.5%	4	12	7min42sec
Challenges	23	20	20	20	454	78.2%	1	4	12min39sec
Icra	118	111	21	98	1434	90.7%	1	10	4min48sec
Tianhan	37	2	3	37	204	100.0%	0	0	1min03sec

Future Works

Accurate Adaptivity Analysis Framework

- Accurate adaptivity definition
- Accurate adaptivity estimation with "path-sensitive reachability-bound algorithm"

Efficient Reachability-bound Algorithm

- More scalable implementation
- Leverage the possible non-terminating in program refining and the ranking function estimating.